

Lien formation complète et simple sur Docker :

- <https://blog.microlinux.fr/formation-docker/>
- <https://docs.docker.com/build/building/best-practices/#from>
- <https://mobaxterm.mobatek.net/download-home-edition.html>
- Putty

Création d'un compte sur Docker Hub

Pour publier une image sur le registre public, il est nécessaire de posséder un compte. Docker propose des comptes gratuits ou payants. Nous nous contenterons du compte gratuit qui permet un nombre de dépôts publics illimités et un dépôt privé.

Procédure de création d'un compte :

Aller sur Docker Hub sur ce lien :

- <https://hub.docker.com>

Cliquez sur « **Sign up for free** » et suivez les instructions pour créer un compte. Le « **username** » sera ensuite obligatoirement utilisé pour préfixer le nom de l'image déployée.

Il suffit ensuite de valider le compte via le mail reçu et de se connecter.

Un dépôt (**repository**) est un ensemble d'images avec le même nom, mais potentiellement des étiquettes (tag) différentes.

Remarque :

Il n'est pas nécessaire de configurer le dépôt avant d'envoyer la première image.

Il sera créé automatiquement sur le Docker Hub public. Dans la page de dépôt, vous pourrez ensuite ajouter une description, voir et supprimer les tags, etc.

1. Se connecter à Docker Hub une fois le compte créé avec votre adresse mail ou nom d'utilisateur et entrez votre mot de passe.
2. Cliquez sur « **Repositories** » et cliquez aussi dans la page qui apparaît, votre Repository.
3. Donnez un nom à ce dépôt « **reposimages** » par exemple et cliquez sur « **Create** ».

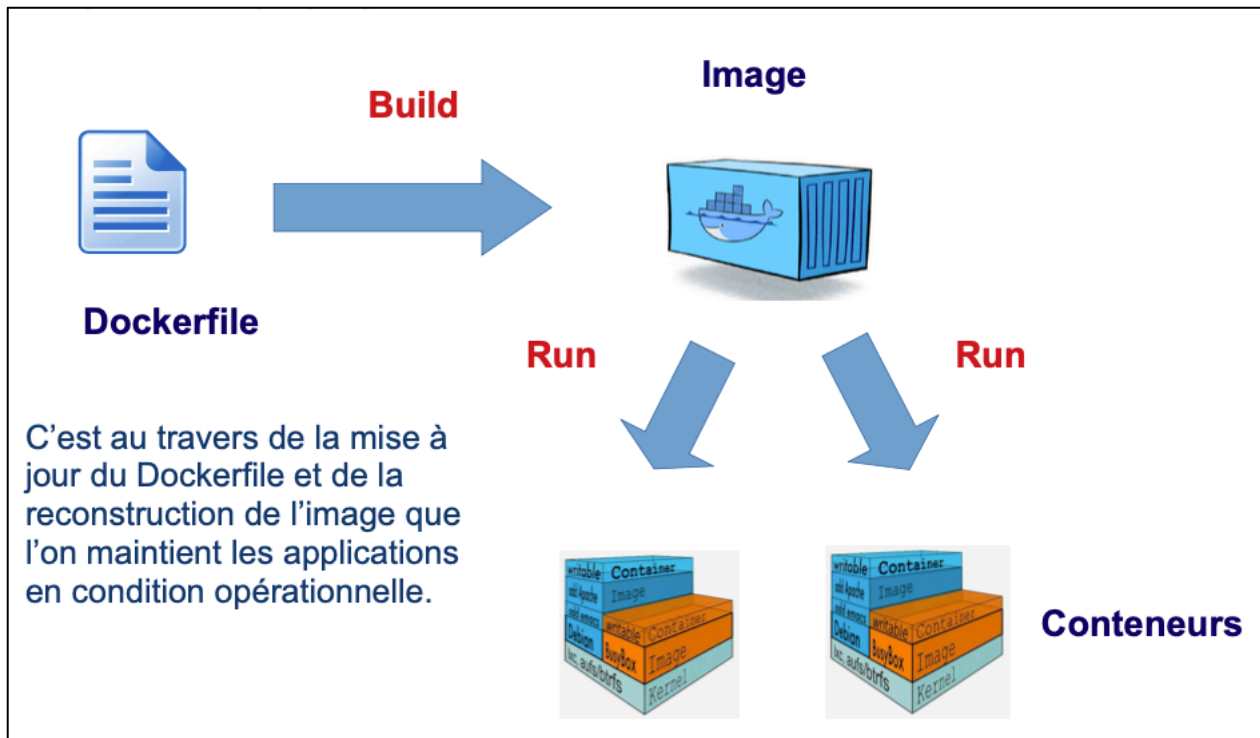
The screenshot shows the 'Create repository' page on Docker Hub. At the top, there are links for 'Repositories' and 'Create', and a status 'Using 0 of 1 private repositories. Get more'. The main section is titled 'Create repository' and contains a 'Repository Name' field with the value 'reposimages'. Below it is a 'Short description' field. A note explains that the description is used for indexing and search. The 'Visibility' section shows two options: 'Public' (which appears in search results) and 'Private' (which is selected, meaning only visible to you). At the bottom right, there are 'Cancel' and 'Create' buttons. On the right side of the form, there is a 'Pushing images' section with instructions on how to push a new image to the repository using the CLI, showing commands like 'docker tag local-image:tagname new-repo:tagname' and 'docker push new-repo:tagname'. It also includes a reminder to replace 'tagname' with the desired image repository tag.

Remarque :

Il est possible de gérer un dépôt Docker avec comme source un dépôt tel que Gitlab ou Github. La plateforme Docker se charge alors automatiquement de la compilation et de la génération de l'image Docker avec une mise à jour du dépôt dès que le code source déposé sur Gitlab ou Github est modifié.

Le fichier DOCKERFILE

Le principe de fonctionnement de **DOCKERFILE** est le suivant :



Docker construit automatiquement les images en lisant, dans l'ordre, les instructions d'un fichier Dockerfile (appelé par défaut « **dockerfile** »). La commande permettant de construire l'image a la structure suivante :

- **docker build [-t <nom_image>[:<tag>]] <dossier_dockerfile>**

Lorsque vous exécutez cette commande, le répertoire de travail est appelé contexte de construction. Par défaut, le fichier « **dockerfile** » est supposé se trouver dedans.

Les étapes de création :

1. Créer un répertoire pour le contexte de construction :

- **mkdir -p ~/contexte/ubuntu-ssh.**

2. Créer et éditer dedans un fichier appelé « **dockerfile** »

On peut utiliser un éditeur en ligne de commande comme **VIM** ou **NANO**, mais vous pouvez aussi de travailler avec **VSCode** ou **VSCodium** ou autre pour avoir plus d'aide dans le débogage et le confort aussi.

Création du fichier DOKERFILE

Choix de la distribution ou services :

Nous allons créer une image basée sur celle de la dernière version TLS d'Ubuntu (Ubuntu 22.04), y installer et configurer le service SSH.

Vous trouverez à cette adresse :

- https://docs.docker.com/develop/develop-images/dockerfile_best-practices/
Les bonnes pratiques pour la construction d'une image.

À noter également que, généralement, les images disponibles sur le registre public sont accompagnées du fichier **Dockerfile** qui a permis de les générer. Ce qui suit ne constitue qu'un exemple simplifié de ce qu'il est possible de faire.

Voici un exemple de fichier, tel que :

- Les indentation présentes dans ce fichier ne sont pas obligatoire comme pour le cas Python
- Le « \> » permet de poursuivre une entrée à la ligne suivante.

```
FROM ubuntu:22.04

LABEL Mainteneur="Prénom Nom [@mail ou URL]"
LABEL description="Ubuntu TLS avec le serveur SSH activé"

# Installation des logiciels
RUN apt update \
    && apt -y --no-install-recommends install \
        openssh-server \
        tzdata \
        locales \
    && rm -rf /var/lib/apt/lists/* \
    && apt clean

# Configuration de la Timezone Europe/Paris et des locales
ENV LANG=fr_FR.UTF-8

RUN rm /etc/localtime \
    && ln -s /usr/share/zoneinfo/Europe/Paris /etc/localtime \
    && dpkg-reconfigure --frontend noninteractive tzdata \
    && sed -i 's/# fr_FR.UTF-8 UTF-8/fr_FR.UTF-8 UTF-8/' /etc/locale.gen \
    && echo "LANG=${LANG}" > /etc/default/locale \
    && dpkg-reconfigure --frontend=noninteractive locales \
    && update-locale LANG=${LANG}

# Création de l'utilisateur SSH
RUN useradd --create-home --shell /bin/bash userssh \
    && echo "userssh:mdpuser" | chpasswd

# Dossier requis par sshd + clés hosts
RUN mkdir -p /run/sshd \
    && ssh-keygen -A

EXPOSE 22

CMD ["/usr/sbin/sshd", "-D"]
```

Explication des mots clés :

FROM : Un fichier Dockerfile commence toujours par la commande FROM, qui permet de définir une image de base à partir de laquelle la nouvelle image sera construite.

Dans notre exemple, l'image de base est une Ubuntu, version 22.04. Si cette image n'est pas présente sur la machine hôte, elle sera téléchargée à partir des registres officiels Docker.

LABEL : Fournit des couples clé/valeur qui serviront de métadonnées décrivant l'image lorsque celle-ci sera diffusée et utilisée. Cette technique permet de communiquer des informations aux utilisateurs de l'image.

ENV : Permet de définir des variables d'environnement exploitables lors de la compilation du Dockerfile et dans les futurs conteneurs (ici pour configurer la date et les locales). Comme nous l'avons déjà vu, les valeurs par défaut définies peuvent ensuite être modifiées via le paramètre « -e » de la commande « **docker run** ».

RUN : permet d'exécuter une ou plusieurs commandes à l'intérieur de l'image. Ici, nous exécutons plusieurs commandes « chaînées » par « && » : mise à jour de la liste des paquets et du système, installation de paquets et nettoyage du gestionnaire de paquets afin que l'image soit un peu plus légère ainsi que dans un second RUN, la mise à jour de la date et des locales, puis la création d'un utilisateur avec l'affectation de son mot de passe et enfin le démarrage du service.

Remarque :

Il est possible et conseillé d'exécuter plusieurs commandes sur une même instruction RUN pour limiter le nombre de layer (chaque commande dans le fichier Dockerfile génère une couche – layer – qui vient se superposer à la couche précédente) et donc la taille de l'image. Nous aurions pu ne faire qu'une seule commande RUN (mais cela nuit à la lisibilité du Dockerfile).

Remarque :

la commande « sed » permet de remplacer une expression par une autre. Ici, il s'agit d'enlever le « # » pour dé-commenter « fr_FR.UTF-8 UTF-8 » dans le fichier « /etc/locale.gen ». On remplace « # fr_FR.UTF-8 UTF-8 » par « fr_FR.UTF-8 UTF-8/ »

EXPOSE : Indique, le ou les ports sur lesquels un conteneur écoute les connexions (on utilise en règle générale le port « connu » pour l'application). Pour un accès externe, les utilisateurs devront exécuter « **docker run** » avec l'option « -p » pour mapper le port.

Remarque :

Nous pouvons considérer ici que la définition de l'image est réalisée : nous avons un service ssh qui pourra potentiellement être utilisé sur le port 22 « exposé ».

CMD : Ce mot clé définit le comportement par défaut au lancement d'un conteneur alors que tout ce qui précédait a permis de définir l'image → Nous décidons ici de lancer le service SSH au premier plan au lancement du conteneur (nous n'aurons pas besoin de le faire avec la commande « docker run »). Ici la syntaxe passe un tableau au format JSON :

- **CMD** [« executable », « param1 », « param2 »...]

Les éléments sont séparés par une virgule et chacun d'eux doit être entre guillemets. À noter qu'il est possible (mais moins conseillé) d'utiliser une autre syntaxe plus simple sous forme de « ligne de commande » :

- « /usr/bin/sshd -D »

Remarque :

S'il y a plusieurs CMD, seule la dernière commande sera exécutée et la commande et les arguments fournis par CMD peuvent être remplacés depuis la ligne de commande lorsque le conteneur docker est lancé : on parle de « surcharge ». Si par exemple on veut seulement faire un « ls » : `docker run <image> ls` avec la commande « ls » qui remplacera le contenu de CMD.

Construction d'une image :

La commande qui permet la construction d'une image est la suivante :

`docker build -t <docker-id>/ubuntu-ssh :1.0 ~/contexte/ubuntu-ssh`

```
mhand@mhanddg:~/contexte/ubuntu-ssh$
mhand@mhanddg:~/contexte/ubuntu-ssh$ nano dockerfile
mhand@mhanddg:~/contexte/ubuntu-ssh$
mhand@mhanddg:~/contexte/ubuntu-ssh$ docker build -t mhandboufala/ubuntu-ssh:v1.0 ~/contexte/ubuntu-ssh
```

Voir l'image :

```
mhand@mhanddg:~/contexte/ubuntu-ssh$
mhand@mhanddg:~/contexte/ubuntu-ssh$ docker images
```

IMAGE	ID	DISK	USAGE	CONTENT	SIZE	EXTRA
mhandbouf/nginx-image:v1	0a3986f700fe	224MB		71.7MB		U
mhandbouf/nginx-test:v2	0a3986f700fe	224MB		71.7MB		U
mhandboufala/ubuntu-ssh:v1.0	f63629288004	165MB		39.6MB		
monnginx:latest	0a3986f700fe	224MB		71.7MB		U
nginx:latest	553f64aecd3	228MB		62.6MB		U
servwebnginx/ubuntu3:ssh	f02804b16f15	444MB		126MB		
servwebnginx/ubuntu:ssh	a1b0c279d5b4	444MB		126MB		U
servwebnginx1/ubuntu4:ssh	001392b62b43	444MB		126MB		

```
mhand@mhanddg:~/contexte/ubuntu-ssh$
```

Lancement d'un conteneur associé à cette image, pas de démarrage du SSh, car prévu dans dockerfile.

```
mhand@mhanddg:~/contexte/ubuntu-ssh$
mhand@mhanddg:~/contexte/ubuntu-ssh$ docker run -d -p 192.168.1.232:2222:22 --name nouvServeurSSH mhandboufala/ubuntu-ssh:v1.0
ee2fec0ac5e4d879b9869516e6179e61b828ef268ea38d2d80858fca837c41fe
mhand@mhanddg:~/contexte/ubuntu-ssh$
mhand@mhanddg:~/contexte/ubuntu-ssh$
```

Test de fonctionnement : Pour cela, il faut se connecter avec le login/mot de passe → « **userssh/mdpuser** » comme ceci :

- `ssh userssh@192.168.1.232 -p 2222`

```

mhand@mhanddg:~/contexte/ubuntu-ssh$
mhand@mhanddg:~/contexte/ubuntu-ssh$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
733533561cda   mhandboufala/ubuntu-ssh:v1.1       "/usr/sbin/sshd -D"     13 seconds ago Up 12 seconds  192.168.1.232:2222->22/tcp  nouvServeurSSH1
mhand@mhanddg:~/contexte/ubuntu-ssh$ ssh userssh@192.168.1.232 -p 2222
The authenticity of host '[192.168.1.232]:2222 ([192.168.1.232]:2222)' can't be established.
ED25519 key fingerprint is SHA256:LIyxjYRRkDgFP7F5xRLTaRVF52NuxKYYp1yY+Qt96w.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[192.168.1.232]:2222' (ED25519) to the list of known hosts.
userssh@192.168.1.232's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-163-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

userssh@733533561cda:~$

```

Publication d'une image sur le DOCKER HUB (REGISTRE PUBLIC)

Revoir notre image afin de s'assurer du nom et du tag avec : `docker ps`

Pour envoyer l'image maintenant, il est nécessaire de s'authentifier en console :

- `docker login -u <nom utilisateur distant sur docker hub>`
- n'oubliez pas le mot de passe

```

mhand@mhanddg:~/contexte/ubuntu-ssh$
mhand@mhanddg:~/contexte/ubuntu-ssh$
mhand@mhanddg:~/contexte/ubuntu-ssh$ docker login -u mhandboufala
Info → A Personal Access Token (PAT) can be used instead.
To create a PAT, visit https://app.docker.com/settings

Password:

WARNING! Your credentials are stored unencrypted in '/home/mhand/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
mhand@mhanddg:~/contexte/ubuntu-ssh$ docker ps

```

Envoyez maintenant vers DOCKER HUB :

```

mhand@mhanddg:~/contexte/ubuntu-ssh$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
733533561cda   mhandboufala/ubuntu-ssh:v1.1       "/usr/sbin/sshd -D"     25 minutes ago Up 25 minutes  192.168.1.232:2222->22/tcp  nouvServeurSSH1
mhand@mhanddg:~/contexte/ubuntu-ssh$ docker commit nouvServeurSSH1
sha256:8cb83391ee44b626741eac34ddf714bfaf1422e610241d793f9790fdc3bc7d21
mhand@mhanddg:~/contexte/ubuntu-ssh$
mhand@mhanddg:~/contexte/ubuntu-ssh$ docker push mhandboufala/ubuntu-ssh:v1.1
The push refers to repository [docker.io/mhandboufala/ubuntu-ssh]
e284debfd17c: Pushed
3bd2967914b1: Pushed
7e49dc6156b0: Pushed
005437ff220b: Pushed
d233140f6bd1: Pushed
e68376985efb: Pushed
v1.1: digest: sha256:e9039645f99befb9e02581b9e9a8f297d41aa9bd183a0c2fcc05c077d52f3ee5 size: 856
mhand@mhanddg:~/contexte/ubuntu-ssh$

```

Aller dans DOCKER HUB :

On voit bien que l'image est déployée.

Si vous cliquez sur l'image, vous allez voir ces détails. L'image est maintenant déployée sur le registre officiel et peut être utilisée par n'importe qui et à partir de n'importe quel système.

The screenshot shows the Docker Hub interface for the repository `mhandboufala/ubuntu-ssh`. The page includes a header with navigation links, a repository overview section with a table of tags, and a sidebar with additional information.

Repository overview

Tag	OS	Type	Pulled	Pushed
v1.1	linux/amd64	Image	less than 1 day	6 minutes

Repository overview (INCOMPLETE)

An overview describes what your image does and how to run it. It displays in [the public view of your repository](#) once you have pushed some content.

[Add overview](#)

Docker commands

To push a new tag to this repository:

```
docker push mhandboufala/ubuntu-ssh:tagname
```

[Public view](#)

buildcloud

Build with **Docker Build Cloud**

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without managing infrastructure.

[Go to Docker Build Cloud](#)

Exercice : à rendre

L'objectif est de créer une image très simple (login/debian12-apache2:1.0) permettant de déployer des pages Web statiques via le logiciel Apache puis, une fois opérationnelle, la publier sur le registre officiel.

Contraintes particulières :

- L'image doit être basée sur la version 12 de Debian et non sur l'image « httpd ».
- Le serveur Web écoutera sur le port 80 et doit être accessible sur le port 8008.
- La zone Europe/paris et les locales « françaises » doivent être gérées.
- Une page Web personnalisée doit être intégrée à l'image.

Aide complémentaire :

- Il n'est pas nécessaire de lancer le service apache 2 (comme nous l'avons fait pour ssh) car il est lancé automatiquement après l'installation.
- Le mot clé CMD (CMD ["/usr/sbin/apache2ctl","-D","FOREGROUND"]) définit le comportement par défaut au lancement d'un conteneur : ici le service apache2 sera démarré par défaut au premier plan (nous n'aurons pas besoin de le faire avec la commande « docker run »).

Travail à faire :

1. Créer le fichier dockerfile qui correspond à cette situation
2. Créer l'image correspondante
3. Lancer le conteneur
4. Tester l'image
5. Publier l'image sur le DOCKER HUB et vérifier qu'elle est disponible en ligne